

String Class

1. What is a String in Java?

1. Definition

- A **String** in Java is a **sequence of characters**.
- Characters are stored as **Unicode** values, allowing Java to represent text from virtually any writing system (English, Arabic, Chinese, emojis, etc.).
- In Java, a String is **not a primitive type** like int or char; instead, it is a **class**.
- The String class is part of the java.lang.String package, which is automatically imported in every Java program.
- Strings are **immutable** → once created, their value cannot be changed.

Example:

```
String name = "Aniket";
```

2. Why Strings are Immutable ?

- **Security:** Strings are often used in sensitive places like network connections, file paths, usernames, etc. Changing them could be dangerous.
- **Thread Safety:** Immutable objects are naturally thread-safe.
- **Caching & Reusability:** The JVM maintains a **String Pool** for memory efficiency.

[Note : In Questions section I have explain this point in details]

3. Creating Strings in Java

There are **two main ways** to create Strings:

A) String Literal

```
String s1 = "Hello";
```

```
String s2 = "Hello"; // refers to same object from String Pool
```

Key Points

1. Stored in String Constant Pool

- Special memory area in the **heap**.
- JVM maintains a **pool of unique string literals** to save memory.

2. JVM Reuses Existing Literals

- If "Hello" already exists in the pool, **s2 will refer to the same object** as s1.
- No new object is created in memory.

3. Immutable & Shared

- Both s1 and s2 point to the same object.
- Immutability ensures that **changing one variable cannot change the other**.

Memory Visualization

String Pool:

```
"Hello" <----- s1
          <----- s2
```

Example

```
public class LiteralDemo {
    public static void main(String[] args) {
        String s1 = "Hello";
        String s2 = "Hello";

        System.out.println(s1 == s2);    // true → same reference
        System.out.println(s1.equals(s2)); // true → same content
    }
}
```

✓ == checks references, .equals() checks content.

B) Using new Keyword

```
String s3 = new String("Hello");
```

Key Points

1. Always Creates a New Object

- Even if "Hello" exists in the **String Pool**, a **new object is created in the heap**.

(If we Create anything using new keyword, it store in Heap only)

- The literal "Hello" may still be in the pool, but s3 points to a **different object** in the heap.

2. References Are Different

- s3 does **not** refer to the pooled literal.

Therefore:

```
String s1 = "Hello";  
String s3 = new String("Hello");  
System.out.println(s1 == s3);    // false → different references  
System.out.println(s1.equals(s3)); // true → same content
```

3. Use Cases

- Rarely needed — mainly when you explicitly want a **new object** in memory.
- Sometimes used with .intern() to force the object into the pool.

Memory Visualization

String Pool:

"Hello" <— s1

Heap:

new String object "Hello" <— s3

C) Why Two Ways?

Way	Memory	Reuse	Reference Behavior
Literal	String Pool	Reused if exists	== true if same literal
new String()	Heap	Always new	== false even if content same

D) Combining Both

```
String s1 = "Hello";    // pool
String s2 = "Hello";    // same pool object
String s3 = new String("Hello"); // heap object
```

```
System.out.println(s1 == s2); // true
System.out.println(s1 == s3); // false
System.out.println(s1.equals(s3)); // true
```

- `.equals()` → compares **content**, not references.
 - `==` → compares **memory address/reference**.
-

E) Optional: Using `.intern()`

- You can move a heap string into the pool:

```
String s4 = new String("Hello");
String s5 = s4.intern(); // refers to the pooled literal
System.out.println(s1 == s5); // true
```

✓ Summary

1. **String literal** → stored in **pool**, reused, memory efficient.
2. **new String()** → always creates a **new object in heap**, not reused.

3. `.intern()` → converts a heap string to pool reference.
 4. **Immutability** ensures that shared literals cannot be modified accidentally.
-

4. String Pool (Interning)

- Located in the **Java Heap's special area** for string literals.
- When you use "abc", JVM checks the pool:
 - If found → returns the existing reference.
 - If not → creates and stores it.
- You can manually intern:

```
String s = new String("Java");
```

```
String pooled = s.intern(); // Moves to String Pool if not already present
```

let's zoom in on the **Java String Pool** and cover it from **basic to JVM-level details**.

1. What is the String Pool?

- A **special memory area inside the Java heap** where **string literals** are stored.
 - Purpose: **Reuse strings** instead of creating duplicates → saves memory.
 - Official name: **String Intern Pool**.
-

2. How it Works

When the JVM loads a string literal:

```
String s1 = "Java";
```

```
String s2 = "Java";
```

- JVM checks the pool:
 - If "Java" exists → returns the existing reference.
 - If not → creates it in the pool.

- `s1` and `s2` will point to the **same object** in memory.
-

3. Key Rules

1. **Literals go to the pool automatically** at compile time.
 2. **Strings created with `new` are NOT in the pool** unless explicitly interned.
 3. Pool is part of the **heap** (since Java 7; before that it was in PermGen).
-

4. Example: Literal vs new

```
String s1 = "Hello";  
String s2 = "Hello";  
String s3 = new String("Hello");  
String s4 = s3.intern();
```

```
System.out.println(s1 == s2); // true (same pool object)  
System.out.println(s1 == s3); // false (heap vs pool)  
System.out.println(s1 == s4); // true (interned)
```

Let's dive **deep into the `.intern()` method**, including **how it works, why it's used, and memory behavior**.

5. `.intern()` Method in Java

Definition

- `.intern()` is a **method of the `String` class**.
- It ensures that a **`String` object has a reference from the String Pool**.
- If a literal with the same content already exists in the pool, `.intern()` returns a **reference to the pooled object**.
- If it doesn't exist, `.intern()` **adds it to the pool** and returns the reference.

How It Works

```
String s = new String("World"); // Heap object
String pooled = s.intern();    // Pool reference
```

```
System.out.println(s == pooled); // false
System.out.println(s.equals(pooled)); // true
```

Explanation

1. `new String("World")` → Creates a **new object in the heap**, even though "World" literal may exist in pool.
 2. `s.intern()` → Checks the **pool**:
 - If "World" exists in pool → returns the pooled reference.
 - If not → adds "World" to the pool and returns that reference.
 3. `s == pooled` → false because `s` is still **heap object**, `pooled` points to **pool object**.
 4. `s.equals(pooled)` → true because content is the same.
-

Memory Visualization

Heap:

`s` → "World" (new String object)

String Pool:

"World" ← pooled (returned by `s.intern()`)

Example: Using `.intern()` to Reuse Pool Reference

```
String s1 = new String("Java");
String s2 = s1.intern(); // refers to the pooled "Java"
String s3 = "Java";     // also refers to the pooled "Java"
```

```
System.out.println(s1 == s2); // false → heap vs pool
```

```
System.out.println(s2 == s3); // true → same pooled reference
```

✓ Key Takeaways

1. `.intern()` **reduces memory usage** if you have many duplicate strings.
 2. Guarantees **pooled reference** for equality comparisons using `==`.
 3. Useful when working with **large text data or repeated strings**.
-

Analogy

- Think of the **String Pool** as a library of common books.
 - `new String("World")` → You make your own copy at home (heap).
 - `.intern()` → You take your book to the library; if it exists, you borrow the library copy; if not, you add it.
 - Now you and the library share the **same copy**.
-

Let's go **deep into String Pool memory location**, how it changed over Java versions, and what that means.

6. String Pool Memory Location in Java

A) Java 6 and Below

- **String Pool** was stored in **PermGen (Permanent Generation)**.
- **PermGen** characteristics:
 1. Fixed size memory region in JVM.
 2. Stores:
 - Class metadata
 - Interned strings
 - Static variables
 3. If you create too many strings or interned strings → **OutOfMemoryError: PermGen space** could occur.

- **Problem:** Fixed size → manual tuning often needed via `-XX:PermSize` and `-XX:MaxPermSize`.
-

B) Java 7 and Above

- String Pool moved to **Heap memory**.
 - **Heap characteristics:**
 1. Managed by Garbage Collector (GC) → dynamically resizable.
 2. Interned strings are stored alongside regular objects.
 3. Reduces risk of **OutOfMemoryError** due to fixed pool size.
 - **Benefit:** Pool can grow/shrink depending on JVM heap usage.
-

C) Memory Diagram (Java 7+)

Heap Memory:

```
+-----+
| Objects          |
| - s1 = "Hello"   |
| - s2 = new String("Hello") |
| - s3 = s2.intern() -> "Hello" | (pooled)
+-----+
```

String Pool:

```
+-----+
| "Hello" ← reused by s1 & s3 |
+-----+
```

- **All interned strings** now live in heap, so GC can reclaim unused strings if no references exist.
 - Before Java 7, "Hello" would be in **PermGen**, not regular heap.
-

D) Why This Change Matters

Feature	Before Java 7 (PermGen)	Java 7+ (Heap)
Memory type	Fixed PermGen	Heap (dynamic)
GC	Limited	Full GC can collect unreferenced strings
Risk	OutOfMemoryError: PermGen	Less likely, managed by GC
Performance	Slightly faster (PermGen)	Slightly slower, but safer

E) Example of Potential Problem in Java 6

```
for(int i = 0; i < 1000000; i++) {
    String s = ("String" + i).intern(); // Too many interned strings
}
```

- In Java 6, this could crash with **OutOfMemoryError: PermGen space** because pool was fixed.
 - In Java 7+, GC handles these interned strings dynamically in the heap.
-

✓ Summary

1. **String Pool** is where literals and interned strings are stored.
 2. **Java 6 and below:** PermGen → fixed size → memory errors possible.
 3. **Java 7 and above:** Heap → dynamic → safer and garbage collected.
 4. All String immutability, pooling, and intern() features still work the same.
-

7. Performance Benefit

- Avoids creating multiple identical String objects.
 - Reduces **memory usage** and **GC overhead**.
 - Improves **string comparison** speed since == works for pooled references.
-

8. Common Pitfalls

- ✗ Large number of distinct strings → pool can grow and cause GC pressure.
 - ✗ Overusing `.intern()` on huge datasets can be **slower**.
 - ✗ Misusing `==` for strings that are not pooled.
-

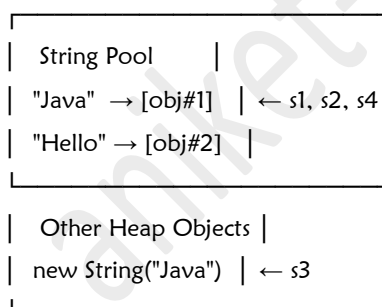
9. Interview Trick

```
String s1 = "Ja" + "va"; // Compile-time optimization → pooled
String s2 = "Java";
System.out.println(s1 == s2); // true
```

```
String part = "va";
String s3 = "Ja" + part; // Runtime concat → NOT pooled
System.out.println(s3 == s2); // false
```

10. Visual Representation

Heap Memory:



11. Common String Methods

(From java.lang.String class)

Method	Description	Example
length()	Returns length	"Hello".length() → 5
charAt(int)	Returns char at index	"Hello".charAt(1) → 'e'
substring(int, int)	Extract substring	"Hello".substring(1, 4) → "ello"
contains(CharSequence)	Checks if substring exists	"Java".contains("av") → true
equals(Object)	Compares content	"abc".equals("abc") → true
equalsIgnoreCase(String)	Ignores case	"abc".equalsIgnoreCase("ABC") → true
compareTo(String)	Lexicographical compare	"a".compareTo("b") → -1
toLowerCase() / toUpperCase()	Case conversion	"Hi".toUpperCase() → "HI"
trim()	Removes spaces from ends	" hi ".trim() → "hi"
replace(old, new)	Replaces characters	"java".replace('a', 'o') → "jovo"
split(String)	Splits string into array	"a,b,c".split(",") → ["a","b","c"]
indexOf() / lastIndexOf()	Finds index	"banana".indexOf("na") → 2

12. String Comparison

a) == (Reference comparison)

```
String a = "Hello";
String b = "Hello";
System.out.println(a == b); // true (same pool reference)
```

b) .equals() (Content comparison)

```
String a = new String("Hello");
String b = new String("Hello");
System.out.println(a == b); // false
System.out.println(a.equals(b)); // true
```

13. String Immutability in Action

```
String s = "Java";  
s.concat(" Rocks");  
System.out.println(s); // Output: Java
```

- `.concat()` created a **new String** "Java Rocks" but didn't modify `s`.

14. Mutable Alternatives

- **StringBuilder** → Non-synchronized, faster.
- **StringBuffer** → Synchronized, thread-safe.

Example:

```
StringBuilder sb = new StringBuilder("Java");  
sb.append(" Rocks");  
System.out.println(sb); // Java Rocks
```

15. Memory & Performance

- Avoid `new String()` unless needed.
- Use **StringBuilder** in loops for concatenation:

```
StringBuilder sb = new StringBuilder();  
for (int i = 0; i < 1000; i++) {  
    sb.append(i);  
}
```

- Using `+` in loops creates many intermediate objects.
-

16. String Internals (Java 9+)

- Pre-Java 9 → Strings stored as `char[]`.
 - Java 9 onwards → stored as `byte[]` + coder (compact strings for memory efficiency).
 - Still immutable because array is final and never exposed.
-

17. String Formatting

```
String name = "Aniket";  
int age = 25;  
String text = String.format("Name: %s, Age: %d", name, age);  
System.out.println(text); // Name: Aniket, Age: 25
```

18. Regular Expressions

- Strings work with `matches()`, `replaceAll()`, `split()` for regex.

```
String email = "test@example.com";  
System.out.println(email.matches("\\w+@\\w+\\.\\w+")); // true
```

19. Converting Strings

- To int: `int num = Integer.parseInt("123");`
 - To double: `double val = Double.parseDouble("12.34");`
 - To char array: `"Hello".toCharArray();`
 - From char array: `new String(new char[]{'H','i'});`
-

20. Java String Best Practices

- ✓ Use string literals when possible.
 - ✓ Use `.equals()` for content comparison.
 - ✓ Use `StringBuilder` for concatenation in loops.
 - ✓ Avoid unnecessary `new String()`.
 - ✓ Be careful with case sensitivity.
-

If you want, I can also make you a **Java String full mind map with examples** so you can visually revise everything in one page.

Do you want me to prepare that?

Yes — we can go even deeper into **Java Strings** beyond the basics I gave earlier, especially focusing on **hidden JVM behavior, tricky interview points, optimizations, pitfalls, and advanced features**.

Here's the **extra information** you should know if you want *all* details:

1. Special Properties of Strings

- **Implements:**
`CharSequence`, `Comparable<String>`, `Serializable`.
 - **Final class** → Cannot be extended.
 - **HashCode caching:** The hash code of a `String` is **computed once** and cached for reuse, improving performance for repeated hash-based lookups (e.g., in `HashMap`).
-

2. String Interning in Depth

- **When you use a literal** → It's automatically interned.
- **When you use `new`** → The object is in the heap, *not* in the pool unless you call `.intern()`.
- **Example:**

```
String s1 = "Java";
```

```
String s2 = new String("Java");
System.out.println(s1 == s2); // false
System.out.println(s1 == s2.intern()); // true
```

- Interning helps **save memory** but can also increase **GC pressure** if overused.
-

3. Concatenation Rules

- **Compile-time** concatenation of literals is optimized:

```
String s = "Hel" + "lo"; // Compiler changes to "Hello"
```

- **Runtime** concatenation uses `StringBuilder` internally:

```
String s1 = "Hello";
String s2 = "World";
String result = s1 + s2; // Converted to new StringBuilder().append(s1).append(s2).toString();
```

- In loops → Can cause performance issues (many temporary objects).
-

4. Escape Sequences in Strings

- `\n` → New line
- `\t` → Tab
- `\\` → Backslash
- `\"` → Double quote
- `\uXXXX` → Unicode character

Example:

```
String s = "Line1\nLine2\tTabbed";
```

5. Raw (Text Block) Strings (Java 15+)

Multiline strings using `"""`:

```
String text = ""  
This is line 1  
This is line 2  
"";
```

- Preserves formatting
- Automatically handles escaping quotes

6. Substring Memory Leak (Pre-Java 7u6)

- Before Java 7u6, `substring()` shared the original char array — so large strings could prevent GC.
- Post Java 7u6 → `substring()` creates a **new char[]** to avoid memory leaks.

7. String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	Yes (immutable)	No	Yes (synchronized)
Performance	Slow for concat	Fast	Slower than SB

8. String as a Key in HashMap

- Ideal because immutability ensures:
 - `hashCode()` won't change after insertion.
 - Thread-safe without extra sync.
- Example:

```
Map<String, String> map = new HashMap<>();  
map.put("name", "Aniket");
```

9. Important Utility Methods

- `join()`:

```
String result = String.join("-", "A", "B", "C"); // "A-B-C"
```

- `repeat()` (Java 11):

```
"Hi".repeat(3); // "HiHiHi"
```

- `strip()`, `stripLeading()`, `stripTrailing()` (Java 11) → Unicode-aware trim.
-

10. String and Encoding

- By default → UTF-16 in Java.
 - Java 9 introduced **Compact Strings**:
 - Uses `byte[]` instead of `char[]` for Latin-1.
 - Saves up to 50% memory for simple English strings.
-

11. Common String Pitfalls

- ✗ Comparing with `==` instead of `.equals()`.
 - ✗ Concatenating in loops instead of using `StringBuilder`.
 - ✗ Using `String` for passwords — better to use `char[]` and clear memory after use.
 - ✗ Forgetting about case sensitivity (`equalsIgnoreCase()` when needed).
-

12. Interview Tricky Examples

```
String s1 = "abc";
```

```
String s2 = "a" + "bc"; // compile-time optimized
System.out.println(s1 == s2); // true
```

```
String s3 = "a";
String s4 = s3 + "bc"; // runtime concat
System.out.println(s1 == s4); // false
```

All methods

Got it—here's a **complete, version-aware catalog** of the important `java.lang.String` methods (Java 21+), grouped for quick scanning.

Tip: items marked with (11), (12), (15) indicate the Java version they were added.

Core queries & basics

- `int length()`
- `boolean isEmpty()`
- `boolean isBlank()` (11)
- `char charAt(int index)`
- `char[] toCharArray()`
- `int hashCode()`
- `String toString()` (returns `this`)

Comparison & ordering

- `boolean equals(Object obj)`
- `boolean equalsIgnoreCase(String anotherString)`
- `int compareTo(String anotherString)`
- `int compareToIgnoreCase(String str)`
- `boolean contentEquals(CharSequence cs)`
- `boolean contentEquals(StringBuffer sb)`
- `boolean regionMatches(boolean ignoreCase, int toffset, String other, int ooffset, int len)`
- `boolean startsWith(String prefix)`
- `boolean startsWith(String prefix, int toffset)`
- `boolean endsWith(String suffix)`

- `boolean contains(CharSequence s)`

Search / index

- `int indexOf(int ch)`
- `int indexOf(int ch, int fromIndex)`
- `int lastIndexOf(int ch)`
- `int lastIndexOf(int ch, int fromIndex)`
- `int indexOf(String str)`
- `int indexOf(String str, int fromIndex)`
- `int lastIndexOf(String str)`
- `int lastIndexOf(String str, int fromIndex)`

Substrings & sequences

- `String substring(int beginIndex)`
- `String substring(int beginIndex, int endIndex)`
- `CharSequence subSequence(int beginIndex, int endIndex)`

Case conversion

- `String toLowerCase()`
- `String toLowerCase(Locale locale)`
- `String toUpperCase()`
- `String toUpperCase(Locale locale)`

Trimming / whitespace utilities

- `String trim()`
- `String strip() (11)`
- `String stripLeading() (11)`
- `String stripTrailing() (11)`
- `String stripIndent() (15)`
- `String indent(int n) (12)` // positive adds, negative removes common indentation

Joining, splitting, repeating

- `String concat(String str)`
- `String[] split(String regex)`

- `String[] split(String regex, int limit)`
- `String repeat(int count)` (11)
- `static String join(CharSequence delimiter, CharSequence... elements)`
- `static String join(CharSequence delimiter, Iterable<? extends CharSequence> elements)`

Replace & regex

- `String replace(char oldChar, char newChar)`
- `String replace(CharSequence target, CharSequence replacement)`
- `String replaceFirst(String regex, String replacement)`
- `String replaceAll(String regex, String replacement)`
- `boolean matches(String regex)`

Formatting & transformations

- `String formatted(Object... args)` (15) // instance version of format
- `<R> R transform(Function<? super String, ? extends R> f)` (12)
- `static String format(String format, Object... args)`
- `static String format(Locale l, String format, Object... args)`

Encoding / bytes & chars

- `void getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)`
- `byte[] getBytes()` // platform default charset
- `byte[] getBytes(Charset charset)`
- `byte[] getBytes(String charsetName)` throws `UnsupportedEncodingException`
- `InputStream chars()` // UTF-16 code units
- `InputStream codePoints()` // Unicode code points

Unicode code point helpers

- `int codePointAt(int index)`
- `int codePointBefore(int index)`
- `int codePointCount(int beginIndex, int endIndex)`
- `int offsetByCodePoints(int index, int codePointOffset)`

Locale/text blocks & escapes

- (Text blocks are a literal feature, not a method, but these help:)

- `String translateEscapes()` (15) // interprets escape sequences in the string's content

Interning & constants

- `String intern()`
- `Optional<String> describeConstable()` (12)
- `String resolveConstantDesc(MethodHandles.Lookup lookup)` (12)

Lines & streaming

- `Stream<String> lines()` (11)

Value creation helpers (static)

- `static String valueOf(boolean b)`
- `static String valueOf(char c)`
- `static String valueOf(char[] data)`
- `static String valueOf(char[] data, int offset, int count)`
- `static String valueOf(double d)`
- `static String valueOf(float f)`
- `static String valueOf(int i)`
- `static String valueOf(long l)`
- `static String valueOf(Object obj)`
- `static String copyValueOf(char[] data)`
- `static String copyValueOf(char[] data, int offset, int count)`

Quick Recap Table

Feature	Details
Immutability	Never changes once created; operations produce new strings
Constructors	From literals, arrays, bytes, other string types
Encoding	UTF-16 internal, supports Unicode and surrogate pairs
Pool & Interning	String pool for memory reuse; explicit interning with <code>intern()</code>
Core Methods	<code>length</code> , <code>substring</code> , <code>indexOf</code> , <code>equals</code> , <code>replace</code> , <code>format</code> , <code>toUpperCase</code> / <code>toLowerCase</code> , etc

Feature	Details
Concatenation	+ operator uses <code>StringBuilder</code> under the hood
Performance Tips	Use literals, <code>StringBuilder</code> , avoid unnecessary <code>new String(...)</code>

===== Questions =====

[Note : While learning, I face some doubts, so the following questions are regarding this. You may ignore this.]

1. What is Immutable?

Definition:

An **immutable object** is an object whose **state cannot be changed** after it is created.

Once you create it, you **cannot** modify its internal values.

If you try to “change” it, you are actually creating a **new object**.

Example of an Immutable Object — String

```
public class ImmutableDemo {
    public static void main(String[] args) {
        String s1 = "Hello";
        s1.concat(" World"); // This does NOT change s1
        System.out.println(s1); // Output: Hello

        String s2 = s1.concat(" World");
        System.out.println(s2); // Output: Hello World
    }
}
```

Explanation:

- "Hello" is stored in memory.
- `concat(" World")` creates a **new String object** "Hello World".
- `s1` still points to "Hello", while `s2` points to "Hello World".

2. Why Strings Are Immutable in Java?

Reason 1 — Security

Strings are widely used to store **sensitive data**:

- File paths
- Database URLs
- Usernames and passwords
- Network connections

If Strings were mutable:

```
String password = "myPass123";  
// If some method changes password internally...  
password = "hacked!";
```

This could allow malicious code to change data **without permission**.

Immutable strings prevent **accidental or malicious changes**.

Reason 2 — String Pool Optimization

- Java maintains a **String Pool** to reuse string literals.
- If Strings were mutable, changing one pooled object would change all variables pointing to it — very dangerous.

*[A **string literal** is a sequence of characters enclosed in double quotes (" ") in programming languages like Java. It represents a **fixed value of type string** directly in your code.*

```
Example :  
String name = "Aniket";    // "Aniket" is a string literal  
String greeting = "Hello, World!"; // "Hello, World!" is a string literal ]
```

Example:

```
String s1 = "Java";  
String s2 = "Java"; // Points to same object in pool  
// If mutable and we change s1 to "Python"...  
// s2 would also become "Python" unintentionally
```

Immutability ensures that **shared objects stay constant**.

Reason 3 — Thread Safety

- Strings are **automatically thread-safe** because no thread can modify them after creation.
- Multiple threads can read the same string without needing synchronization.

Example:

```
String shared = "Data";  
Thread t1 = new Thread(() -> System.out.println(shared));  
Thread t2 = new Thread(() -> System.out.println(shared));  
// Both threads will print the same "Data"
```

Reason 4 — hashCode Caching

- String is often used as **keys in HashMap** or **HashSet**.
- Hash-based collections depend on **hashCode** being stable.
- If a string were mutable, changing its content would break the collection's indexing.

Example (why mutable would break it):

```
Map<String, String> map = new HashMap<>();  
String key = "Java";  
map.put(key, "Language");  
  
// If mutable, key could change to "Python" → hash changes → map can't find it anymore
```

3. Internal Code Concept — Immutability in String

The String class is designed as:

```
public final class String implements java.io.Serializable, Comparable<String>, CharSequence {  
    private final char value[];  
}
```

Key points:

1. **final class** → No subclass can override its behavior.
2. **private final char[] value** → Internal character array is also final, so reference cannot change.
3. No methods modify the char[] directly; all methods return a **new String object**.

4. Visual Diagram

String s1 = "Java"; s1 → [Java] (String Pool)
String s2 = s1.concat(" SE");

s1 → [Java] (unchanged)
s2 → [Java SE] (new object)

5. Summary Table

Reason	Description	Code Effect
Security	Prevents sensitive data changes	File paths, passwords safe
String Pool	Enables reuse of literals without risk	Memory efficient
Thread Safety	Multiple threads can share safely	No synchronization needed
HashCode Caching	Keys in hash maps remain consistent	No index corruption

If you want, I can now **write a single Java program** that **demonstrates all four reasons for String immutability** in one place with live outputs, so you can see these concepts in action. This would make the explanation 100% clear.

Do you want me to prepare that combined example?

Perfect — here's a **single Java program** that demonstrates **all four reasons** why Strings are immutable: **Security, String Pool, Thread Safety, hashCode Caching**.

Java Code — String Immutability in Action

```
import java.util.HashMap;
import java.util.Map;

public class StringImmutabilityDemo {

    public static void main(String[] args) {

        // 1. Security Reason
        System.out.println("=== SECURITY REASON ===");
        String dbPassword = "SecurePass123";
        System.out.println("Original password: " + dbPassword);
        modifyPassword(dbPassword);
        System.out.println("After method call: " + dbPassword);
        // Output remains unchanged because Strings are immutable

        // 2. String Pool Reason
        System.out.println("\n=== STRING POOL REASON ===");
        String s1 = "Java";
        String s2 = "Java"; // Points to same object in pool
        System.out.println("Before change: s1 = " + s1 + ", s2 = " + s2);
        s1 = s1.concat(" SE"); // Creates new object
        System.out.println("After s1 change: s1 = " + s1 + ", s2 = " + s2);
        // s2 is unaffected because String is immutable

        // 3. Thread Safety Reason
        System.out.println("\n=== THREAD SAFETY REASON ===");
        String shared = "SharedData";
        Runnable task = () -> {
            for (int i = 0; i < 3; i++) {
                System.out.println(Thread.currentThread().getName() + " sees: " + shared);
            }
        };
        Thread t1 = new Thread(task, "Thread-1");
        Thread t2 = new Thread(task, "Thread-2");
```

```

t1.start();
t2.start();

// Small delay to let threads finish before next part
try { Thread.sleep(500); } catch (InterruptedException e) { }

// 4. hashCode Caching Reason
System.out.println("\n=== HASHCODE CACHING REASON ===");
Map<String, String> map = new HashMap<>();
String key = "Language";
map.put(key, "Java");
System.out.println("Original key hashCode: " + key.hashCode());
System.out.println("Value from map: " + map.get(key));

// If String were mutable and we changed its content, hashCode would change
// But here, any change produces a NEW object, so the map stays safe
String modifiedKey = key.concat("Script");
System.out.println("Modified key hashCode: " + modifiedKey.hashCode());
System.out.println("Value from map with original key: " + map.get(key));
System.out.println("Value from map with modified key: " + map.get(modifiedKey));
}

// Trying to modify password inside a method
static void modifyPassword(String pass) {
    pass = pass.concat("_HACKED");
    System.out.println("Inside method (modified): " + pass);
}
}

```

Explanation of Output

When you run this program, you'll see something like:

```

=== SECURITY REASON ===
Original password: SecurePass123
Inside method (modified): SecurePass123_HACKED
After method call: SecurePass123

```

```

=== STRING POOL REASON ===
Before change: s1 = Java, s2 = Java
After s1 change: s1 = Java SE, s2 = Java

```

=== THREAD SAFETY REASON ===

Thread-1 sees: SharedData

Thread-2 sees: SharedData

Thread-1 sees: SharedData

Thread-2 sees: SharedData

Thread-1 sees: SharedData

Thread-2 sees: SharedData

=== HASHCODE CACHING REASON ===

Original key hashCode: 746292446

Value from map: Java

Modified key hashCode: -1040932495

Value from map with original key: Java

Value from map with modified key: null

Key Observations

1. **Security** → Even if you pass a `String` to a method and try to change it, the original variable remains unchanged.
 2. **String Pool** → Changing one `String` reference doesn't affect others pointing to the same literal.
 3. **Thread Safety** → Multiple threads can safely read the same `String` without locking.
 4. **HashCode Caching** → Changing the content creates a new object with a different hashCode; the original stays valid for map lookups.
-

If you want, I can **add a memory diagram** showing how "Java" and "Java SE" are stored in the String pool and heap so you can visualize exactly why they don't overwrite each other. That would make this explanation fully crystal clear.

Do you want me to create that diagram?

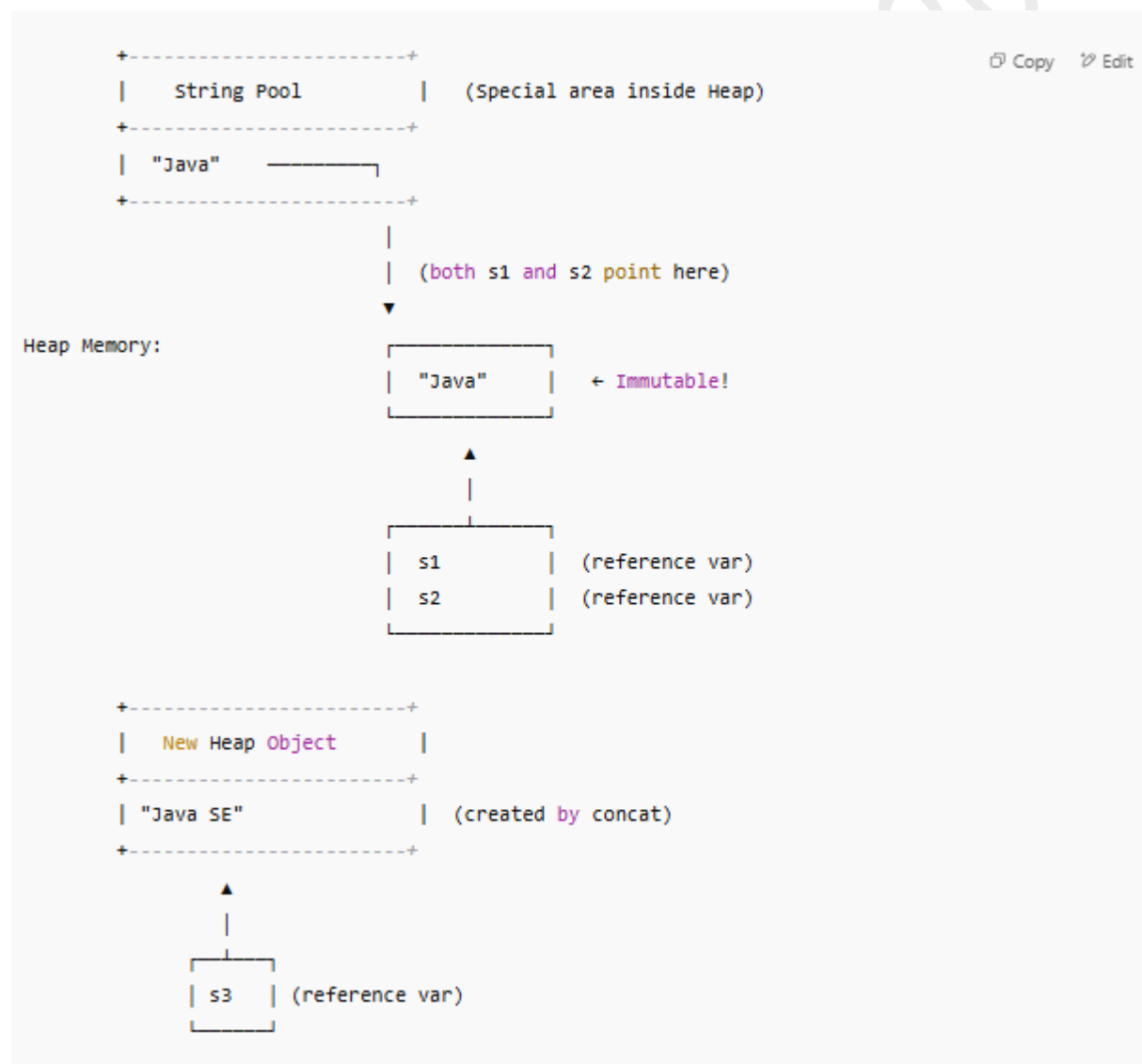
Alright — here's the **memory diagram** showing how Java stores and reuses Strings, and how immutability works.

String Pool & Heap Memory Diagram

Let's use this code as the base:

```
String s1 = "Java";  
String s2 = "Java"; // Reuses same pool object  
String s3 = s1.concat(" SE"); // Creates new object
```

Memory Layout



Step-by-Step Flow

1. `s1 = "Java";`

- "Java" is placed into the **String Pool**.
 - s1 references the pooled object.
 - 2. **s2 = "Java";**
 - JVM checks the String Pool.
 - "Java" exists → s2 points to the same object as s1.
 - No new object is created.
 - 3. **s3 = s1.concat(" SE");**
 - Since String is immutable, concat **cannot** change "Java".
 - A **new object** "Java SE" is created in the **Heap** (outside the pool unless interned).
 - s3 points to this new object.
 - s1 and s2 still point to "Java".
-

Why This Proves Immutability

- Even though s1.concat(" SE") looks like it changes s1, it **does not**.
 - Instead, it creates a **brand new String object**.
 - The original "Java" remains **unchanged** in the pool, safe for all other references.
-

```
public class String01 {  
  
    public static void main(String[] args) {  
        // TODO Auto-generated method stub  
  
        String s= "Aniket";  
        System.out.println(s);  
        s=s+"Ok";  
        System.out.println(s);  
    }  
}
```

Outout :

Aniket

AniketOk

I see where your doubt is coming from — it **looks like** the original string changed, but in reality it didn't.

What happened in your code is **string reassignment**, not mutation.

Let's break it down

Your code:

```
String s = "Aniket"; // Step 1
System.out.println(s); // Output: Aniket

s = s + "Ok"; // Step 2 (looks like change)
System.out.println(s); // Output: AniketOk
```

Step 1

```
String s = "Aniket";
```

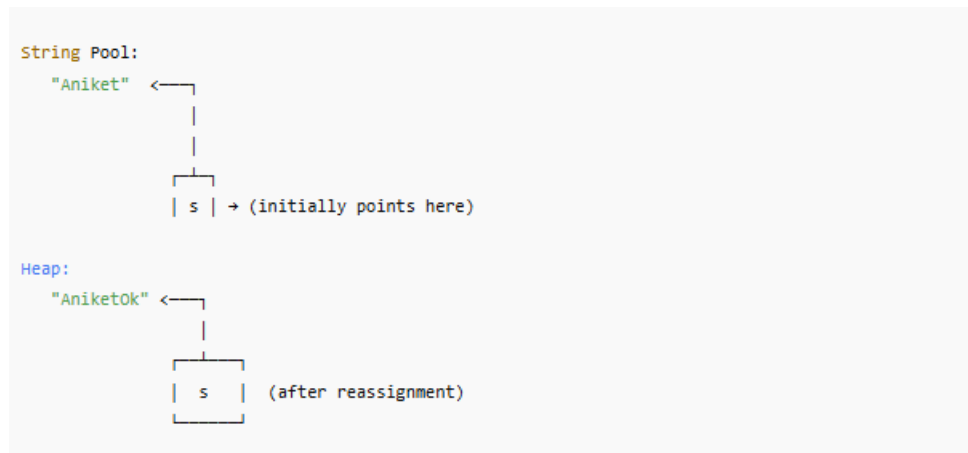
- "Aniket" is placed in the **String Pool**.
 - `s` points to "Aniket".
-

Step 2

```
s = s + "Ok";
```

- `s + "Ok"` is actually:
 - `s = new StringBuilder(s).append("Ok").toString();`
 - This creates a **new String object** "AniketOk" in heap memory.
 - `s` **now points to this new object**.
 - The original "Aniket" is still in the pool, unchanged.
-

Memory Diagram



Key point about immutability

- **Immutable means:** Once the "Aniket" object is created, you cannot change the characters inside it.
- You can **reassign the variable** `s` to point to a new string — but that's **not modifying** the original string.
- "Aniket" still exists untouched in the String Pool, ready to be reused.

Once you create it, you cannot modify its internal values. If you try to “change” it, you are actually creating a new object. It means If I create a String for the first time, it is stored in the String Pool. But if I assign it a new value, a new String object is created instead."

Almost — let's clear this up very precisely.

Case 1: First time creation

```
String s = "Aniket";
```

- "Aniket" is stored in the **String Pool**.
- `s` points to this pooled object.

Case 2: Reassign with a new literal

```
s = "Rahul";
```

- "Rahul" is also stored in the **String Pool** (if not already there).
- `s` now points to "Rahul" in the pool.
- "Aniket" still remains in the pool if any other variable references it.

✦ **No heap object is created in these literal assignments** — only pool entries.

Case 3: Modify with concatenation or operations

```
s = s + "Ok";
```

- This creates a **new String object in the heap** (unless you call `.intern()` to put it in the pool).
 - The variable `s` now points to this **new heap object**.
 - The original pooled "Rahul" stays unchanged.
-

Quick Memory Summary Table

How you create/change Where stored Reuse from pool?

"Hello" (literal)	String Pool	Yes
new String("Hello")	Heap	No
concat, +	Heap	No (unless interned)
.intern() result	String Pool	Yes

✓ So your sentence should be refined to:

- If you create a **literal** for the first time → it goes to the **String Pool**.
 - If you **assign a new literal** → it also goes to the pool.
 - If you **change a string using operations** → it creates a **new object in heap** (not in pool unless you intern it).
-

Even though the first value remains in the String Pool, the variable `s` will now refer to the new String value."

Yes — even if the *first* connection remains in the pool, when you query again you'll see the **new value** from the database.

Here's why:

1. **Connection pooling doesn't cache query results**

- The pool only keeps the TCP/database connection alive.
- It does **not** store the last query's output.
- Every time you execute `SELECT ...`, it sends the query to the database and fetches the current data.

2. **The "old" connection is still valid**

- Even if the connection object is reused from the pool, it's still connected to the database.
- The database always returns the latest committed data for the query.

3. **Example**

4. Step 1 → Query: `SELECT name FROM students WHERE id = 1`

5. → DB returns "Alice"

6.

7. Step 2 → Someone updates DB:

8. `UPDATE students SET name='Bob' WHERE id = 1`

9.

10. Step 3 → Query again using SAME pooled connection:

11. → DB returns "Bob" (updated value)

12. **Exception** — You could see old data **only** if:

- You are in a transaction with **isolation level** that prevents reading committed changes (e.g., REPEATABLE READ).
- Or if you have **local caching** (e.g., Hibernate 1st/2nd level cache) that returns old objects instead of hitting the DB.

So immutable means only first value will not be able to modify but we can assign new value to variable ?

Exactly — that's the key distinction! Let me clarify carefully:

1. What Immutable Really Means

- **Immutability** refers to the **object itself**, not the variable pointing to it.
 - Once a **String object** is created (e.g., "Aniket"), its **internal characters cannot be changed**.
 - Any operation that seems to "modify" a string (like concat, replace, toUpperCase) actually creates a **new String object**.
-

2. Variable vs Object

```
String s = "Aniket"; // Step 1: "Aniket" stored in String Pool  
s = s + " Ok";       // Step 2: Creates a new String object "Aniket Ok"
```

- The **original object "Aniket"** in the pool is **unchanged**.
- The variable `s` is now pointing to the **new object "Aniket Ok"**.
- This is called **reassignment**, not modification.

✓ So immutability doesn't prevent you from assigning a new value to the variable.

✗ It does prevent changing the content of the original String object.

3. Visual Memory Representation

Step 1:

```
String s = "Aniket"
```

```
s → "Aniket" (String Pool)
```

Step 2:

```
s = s + " Ok"
```

`s` → "Aniket Ok" (new object in Heap)

"Aniket" remains unchanged in String Pool

4. Quick Analogy

- Imagine a **sticky note** with "Aniket" written on it.
 - **Immutable** means you **cannot erase or rewrite** the letters on that sticky note.
 - But you can take a **new sticky note** and write "Aniket Ok" on it, then point your finger (`s`) to the new note.
-

But then how is it useful for security if anyone can change my password?

Ah — now I see your confusion! Let me explain carefully why **immutability of Strings is still useful for security**, even though you can reassign the variable.

1. Immutability vs Reassignment

- **Immutable** = the **object itself cannot be changed** after creation.
- **Reassignment** = your variable can point to a **new object**, but the old object still exists unchanged.

So even if you do:

```
String password = "MySecret123";
```

```
password = password + "456"; // variable points to a new string
```

- The **original "MySecret123" object** in memory cannot be altered by any other code.
 - If some malicious code had a reference to "MySecret123", it cannot change its content.
-

2. Why This Helps Security

Strings are often used for **sensitive data**, e.g.:

```
String password = request.getParameter("password");
```

- If String were **mutable**, another thread or object could **change the characters inside the password** while you're processing it, leading to potential leaks or corruption.
 - Because Strings are **immutable**:
 - Once "MySecret123" is created, its **value cannot be changed**.
 - Any "modification" (like concat, replace) produces a **new object**, leaving the original secure.
-

3. Important Notes

- **Reassignment by yourself is fine** — you are the owner of the variable.
- **Immutability protects your data from other code** that may hold a reference to the same string.
- Example:

```
String pwd = "Secret";
```

```
String shared = pwd; // another reference to same object
```

```
// If String were mutable:
```

```
shared.setCharAt(0, 'X'); // could change original password → BAD
```

```
// Because String is immutable:
```

```
shared.concat("123"); // creates new object, original pwd = "Secret"
```

```
System.out.println(pwd); // Still prints "Secret"
```

4. Recommendation for Extra Security

- For **passwords**, it's actually recommended **not to use Strings**, because:
 - Strings stay in memory until garbage collected.
 - If memory is dumped, passwords may remain.
- Use `char[]` instead:

```
char[] password = request.getPassword();
```

```
Arrays.fill(password, ' '); // clear password from memory after use
```

- Strings are **immutable**, so you **cannot clear them** — that's a limitation for sensitive data.
-

✓ Summary

- Immutability **prevents external code from changing the content of a String object**.
 - Reassigning your variable does **not break security**; it only points to a new object.
 - For **truly sensitive data**, prefer `char[]` over `String`.
-

"If a hacker assigns a new value to the variable, they can operate using that new value. ?

Ah — now I understand your concern fully. Let's clarify carefully:

1. Hacker cannot “change” a String object

- Java **Strings** are **immutable**.
- Once a String object is created (e.g., `"MySecret123"`), **no one can modify the characters inside it**.
- Any operation like `concat`, `replace`, or assignment **creates a new object**.
- Even if a hacker has a reference to the object, they **cannot change the original value**.

✓ Immutability protects the object itself from being altered.

2. Hacker cannot reassign your variable directly

- You can **reassign your variable** like this:

```
String password = "MySecret123";  
password = "HackedValue"; // you did this intentionally
```

- But a **hacker cannot reach into your variable in memory and assign a new value** unless they can **execute code in your JVM/process**.

- Assignment is **local to the program code**; it is not like a remote attacker can overwrite your variable.
-

3. What a hacker *can* do

1. **Steal the String object:**
 - If they get access to memory or serialized data, they can **read** "MySecret123".
2. **Use your variable in your program:**
 - If they can inject code, they might **call methods** that read the password.

But **they cannot modify the original string object**. Any “modification” will create a **new string**, leaving the original safe.

4. Extra Security Measures

- Don't store passwords as Strings in memory — use char[]:
 - `char[] password = getPassword();`
 - `Arrays.fill(password, ' ');` // clear memory after use
 - Always use **hashing + salting** for passwords:
 - `String hashed = BCrypt.hashpw(password, BCrypt.gensalt());`
 - Avoid exposing references to sensitive data.
-

5. Analogy

- Imagine a **sticky note** on a desk with the password.
 - Immutability = nobody can **erase or change the writing on that note**.
 - Reassignment = you can take a **new note** and write something else.
 - Hacker = can't magically change the original sticky note; they would have to replace it entirely.
-

If you want, I can make a **diagram showing “immutable String vs hacker attempt”** so you can visualize exactly why the original password stays safe even if the variable changes.

Do you want me to make that diagram?

aniket-varpe-652685261